

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT2-Constraint-Based Problem Solving

Register Number	192524281
Name	KAVIYA.G

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:3

Total Marks:30

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- D1 cannot work Night shift
- D2 must work before D3 (Morning < Afternoon < Night)
- Only one doctor per shift
- D3 cannot work Morning
- All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- Movement allowed: Up, Down, Left, Right
- Cost of each move = 1
- Cannot pass through obstacles
- Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed

- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

*I **KAVIYA.G(192524281)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5
Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6

Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5
----------------------	--	--	-------------------------------	---------------------------	-----

Question 1:

A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- I. **Movement allowed: Up, Down, Left, Right**
- II. **Cost of each move = 1**
- III. **Cannot pass through obstacles**
- IV. **Use Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) using the Backtracking Search Algorithm. Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule.

1. Introduction

Artificial Intelligence (AI) provides intelligent techniques for solving complex decision-making and optimization problems. One of the most effective techniques for solving Constraint Satisfaction Problems (CSPs) is the Backtracking Search Algorithm. A Constraint Satisfaction Problem consists of a set of variables, possible values (domains), and constraints that restrict the combinations of values assigned to the variables. Backtracking Search follows a depth-first search strategy, assigning one variable at a time and verifying whether the assignment satisfies all constraints. If any constraint is violated, the algorithm immediately backtracks to the previous assignment and explores another possible value. This process continues until a complete and valid solution is obtained. Because it systematically eliminates invalid solutions, Backtracking Search is widely used in scheduling, planning, resource allocation, timetable generation, and artificial intelligence applications.

In this problem, the hospital must assign three doctors to three shifts while satisfying all specified scheduling constraints. The objective is to determine a valid schedule without violating any hospital policies.

2. Problem Understanding

The hospital administration needs to prepare a daily duty roster by assigning **three doctors (D1, D2, and D3)** to three work shifts (Morning, Afternoon, and Night). Every doctor must work exactly one shift, and every shift must be assigned to exactly one doctor. The scheduling process must satisfy all operational constraints provided by the hospital. This scheduling problem is modeled as a Constraint Satisfaction Problem (CSP) because each doctor's assignment depends on multiple restrictions. The Backtracking Search Algorithm is applied to systematically search for a valid assignment while checking every constraint after each decision.

4. Problem Formulation

Variables

The variables represent the doctors.

[
X={D1,;D2,;D3}
]

Domain

The domain represents all possible shifts available for each doctor.

Doctor Possible Shifts

D1 Morning, Afternoon

D2 Morning, Afternoon, Night

D3 Afternoon, Night

Since D1 cannot work at Night and D3 cannot work in the Morning, their domains are reduced before the search begins.

5. Constraints

Constraint ID	Description	Purpose
C1	D1 cannot work the Night shift	Personal availability restriction
C2	D2 must work before D3 (Morning < Afternoon < Night)	Maintains scheduling order
C3	Only one doctor can be assigned to one shift	Prevents overlapping assignments
C4	D3 cannot work the Morning shift	Availability restriction
C5	Every doctor must receive exactly one shift	Ensures complete assignment

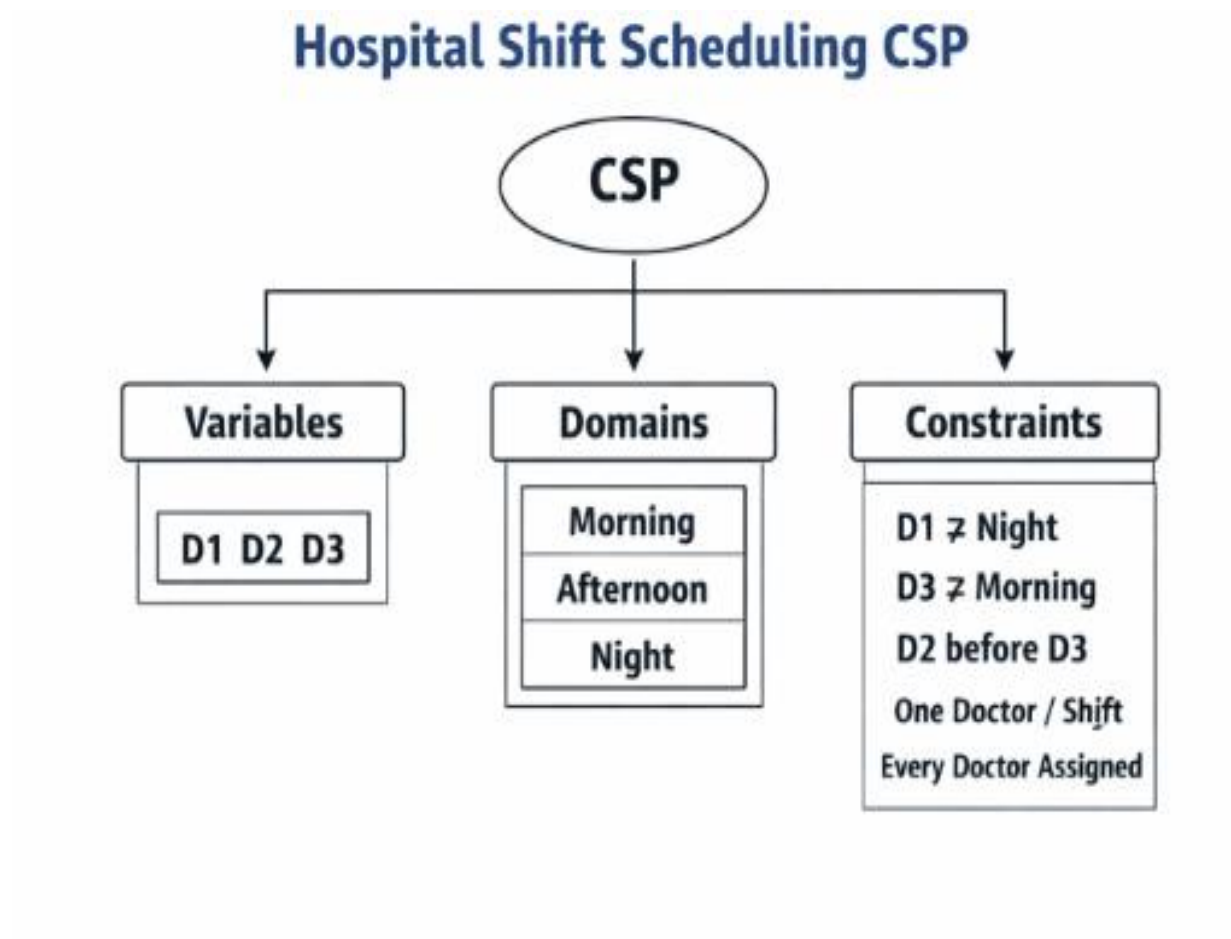
5. Constraint Analysis

The scheduling constraints play an important role in reducing the search space and ensuring a feasible solution.

- **Constraint C1** removes the Night shift from Doctor D1's available choices.
- **Constraint C2** establishes an ordering relationship between Doctors D2 and D3. Therefore, D2 must always be assigned to a shift occurring earlier than D3.
- **Constraint C3** prevents duplicate assignments by allowing only one doctor in each shift.
- **Constraint C4** removes the Morning shift from Doctor D3's available choices.
- **Constraint C5** guarantees that all doctors receive one and only one shift.

By applying these constraints during every stage of the search, invalid assignments are eliminated immediately without exploring unnecessary possibilities.

6. CSP Representation

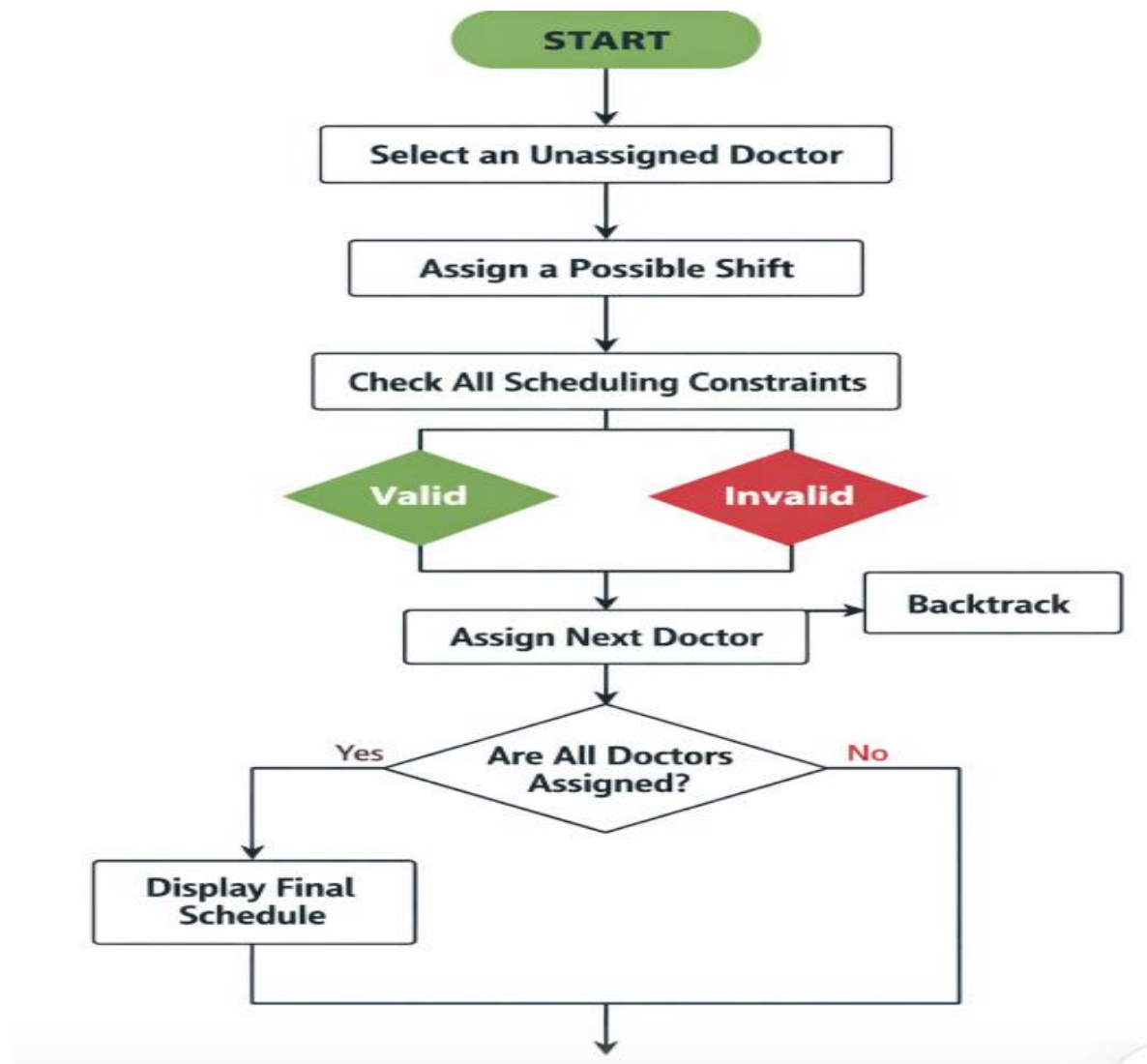


7. Backtracking Search Algorithm

Algorithm Steps

1. Begin with an empty assignment.
2. Select the first unassigned doctor.
3. Assign one available shift.
4. Verify all constraints.
5. If the assignment is valid, continue with the next doctor.
6. If any constraint fails, backtrack to the previous assignment.
7. Try another available shift.
8. Continue until every doctor has been assigned a valid shift.

8. Flowchart



9. Step-by-Step Execution

Initial State

Doctor Assigned Shift

D1 —

D2 —

D3 —

No assignments have been made.

Step 1: Assign Doctor D1

Available shifts:

- Morning
- Afternoon

Choose:

D1 → Morning

Current Assignment

Doctor Shift

D1 Morning

Constraint Checking

- D1 is not assigned Night
- No duplicate assignment

The assignment is valid.

Step 2: Assign Doctor D2

First attempt:

D2 → Morning

Current Assignment

Doctor Shift

D1 Morning

D2 Morning

Constraint Checking

Constraint C3 is violated because two doctors are assigned to the Morning shift.

Result: Backtrack.

Backtracking

Remove

D2 → Morning

Choose the next available shift.

Assign

D2 → Afternoon

Current Assignment

Doctor Shift

D1 Morning

D2 Afternoon

Constraint Check

- Unique shift
- D2 can still be scheduled before D3

Proceed.

Step 3: Assign Doctor D3

Available shifts:

- Afternoon
- Night

First attempt:

D3 → Afternoon

Current Assignment

Doctor Shift

D1 Morning

D2 Afternoon

D3 Afternoon

Constraint C3 is violated because Afternoon is already occupied.

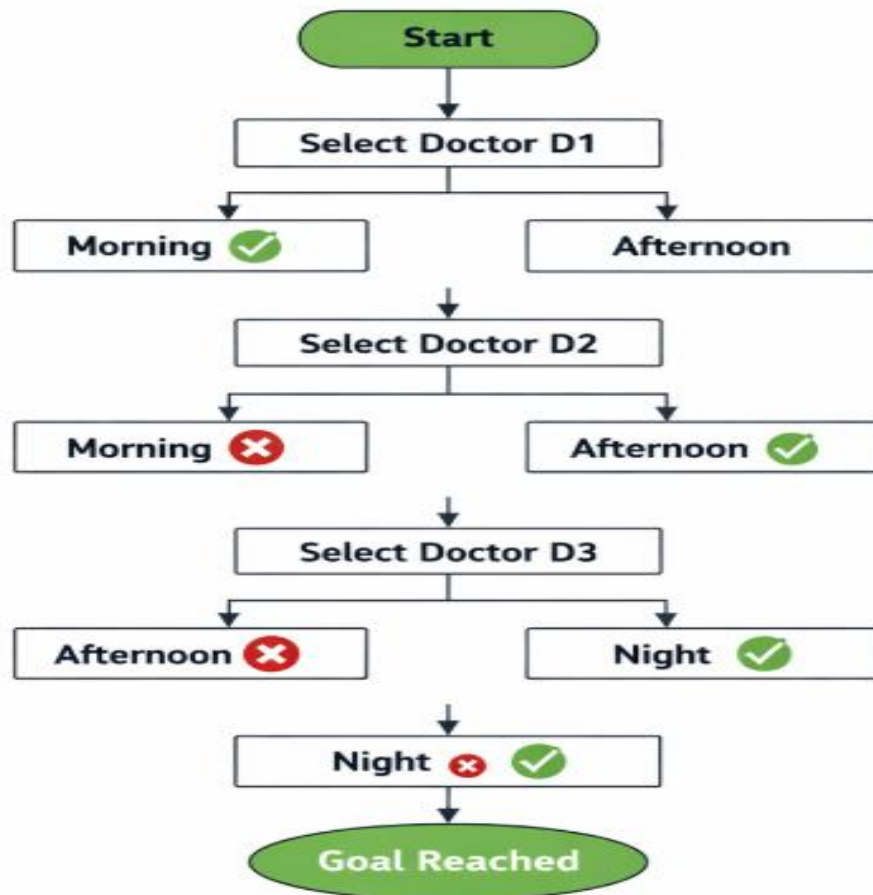
Backtrack.

Choose the remaining shift.

Assign

D3 → Night
 Current Assignment
Doctor Shift
 D1 Morning
 D2 Afternoon
 D3 Night

10. Search Tree



11. Constraint Checking Table

Step	Assignment	Constraint Evaluation	Decision
1	D1 → Morning	Valid	Continue
2	D2 → Morning	Duplicate shift	Backtrack
3	D2 → Afternoon	Valid	Continue
4	D3 → Afternoon	Duplicate shift	Backtrack
5	D3 → Night	All constraints satisfied	Solution Found

12. Final Schedule

Shift Assigned Doctor

Morning D1

Afternoon D2

Night D3

13. Justification of the Solution

The final assignment satisfies every constraint:

- Doctor **D1** is assigned to the Morning shift, satisfying the condition that D1 cannot work at Night.
- Doctor **D2** is assigned to the Afternoon shift, which occurs before Doctor D3's Night shift, satisfying the ordering constraint.
- Doctor **D3** is assigned to the Night shift, satisfying the condition that D3 cannot work in the Morning.
- Every shift is assigned to exactly one doctor, eliminating conflicts or duplicate assignments.
- Every doctor receives exactly one shift, ensuring a complete and feasible schedule.

Therefore, the obtained solution is **correct, feasible, and optimal** because all constraints are satisfied without requiring any additional assignments.

14. Advantages of Backtracking Search

- Guarantees a valid solution if one exists.
- Eliminates invalid assignments through early constraint checking.
- Reduces unnecessary search by backtracking immediately after detecting conflicts.
- Produces an optimal solution for constraint satisfaction problems.
- Suitable for scheduling, planning, resource allocation, and artificial intelligence applications.

15. Applications

Backtracking Search is widely used in:

- Hospital doctor and nurse scheduling
- University examination timetables
- Airline crew scheduling
- Employee shift allocation
- Task scheduling in cloud computing
- Puzzle solving (Sudoku, N-Queens)
- AI planning and resource management

16. Conclusion

The Backtracking Search Algorithm successfully solved the hospital shift scheduling problem by systematically assigning doctors to available shifts while verifying all constraints at every stage of the search. Whenever an invalid assignment was encountered, the algorithm backtracked to the previous decision and explored an alternative assignment. This process continued until a feasible solution satisfying every constraint was obtained. The final schedule D1 assigned to the Morning shift, D2 assigned to the Afternoon shift, and D3 assigned to the Night shift fulfills all scheduling requirements and demonstrates the effectiveness of Backtracking Search in solving real-world Constraint Satisfaction Problems efficiently and accurately.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- v. **Movement allowed: Up, Down, Left, Right**
- vi. **Cost of each move = 1**
- vii. **Cannot pass through obstacles**
- viii. **Use Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Problem Solving Using Breadth-First Search (BFS) Algorithm

Question

A robot operates in a **5 × 5 grid environment**. It must move from the **Start (S)** node to the **Goal (G)** node while avoiding obstacles.

Constraints

- Grid Size = **5 × 5**
- Start Node = **S**
- Goal Node = **G**
- Obstacles block certain cells.
- Allowed movements:
 - Up
 - Down
 - Left
 - Right
- Cost of each movement = **1**
- Manhattan Distance heuristic is given for comparison.
- Apply **Breadth-First Search (BFS)**.
- Show
 - State Space
 - Step-by-step node expansion
 - Queue updates
 - Determine optimal path
 - Calculate total cost

1. Problem Formulation

The objective is to find the shortest path for a robot from the starting location to the goal location in a two-dimensional grid while avoiding obstacles. Since every movement has the same cost, Breadth-First Search (BFS) guarantees the shortest path.

Although the Manhattan Distance heuristic is provided, BFS is an uninformed search algorithm and therefore does not use heuristic values during node expansion. The heuristic is calculated only for analysis and comparison with informed search algorithms such as A* Search.

2. State Space Representation

Grid

C0 C1 C2 C3 C4

R0 S □ X □ □

R1 □ □ X □ □

R2 □ □ □ X □

R3 X □ □ □ □

R4 □ □ □ □ G

Symbols

- **S** = Start
- **G** = Goal
- **X** = Obstacle
- □ = Free Cell

3. Initial State

Start Position

$S = (0,0)$

Goal Position

$G = (4,4)$

Initial Queue

$[(0,0)]$

Visited Nodes

$\{(0,0)\}$

4. Breadth-First Search Algorithm

Algorithm

Step 1: Insert the Start node into the queue.

Step 2: Mark the Start node as visited.

Step 3: Remove the front node from the queue.

Step 4: Check whether it is the Goal node.

Step 5: If Goal is reached, stop.

Step 6: Otherwise, generate all valid neighbouring nodes
(Up, Down, Left, Right).

Step 7: Ignore obstacles and already visited nodes.

Step 8: Insert new nodes into the queue.

Step 9: Repeat Steps 3–8 until the Goal is found.

5. Step-by-Step Node Expansion

Step 1

Expand

(0,0)

Generated Nodes

(0,1)

(1,0)

Queue

[(0,1),(1,0)]

Step 2

Expand

(0,1)

Generated Node

(1,1)

(Right is blocked by an obstacle.)

Queue

[(1,0),(1,1)]

Step 3

Expand

(1,0)

Generated Node

(2,0)

Queue

[(1,1),(2,0)]

Step 4

Expand

(1,1)

Generated Node

(2,1)

Queue

[(2,0),(2,1)]

Step 5

Expand

(2,0)

No new valid nodes.

Queue

[(2,1)]

Step 6

Expand

(2,1)

Generated Nodes

(2,2)

(3,1)

Queue

[(2,2),(3,1)]

Step 7

Expand

(2,2)

Generated Node

(3,2)

Queue

[(3,1),(3,2)]

Step 8

Expand

(3,1)

Generated Node

(4,1)

Queue

[(3,2),(4,1)]

Step 9

Expand

(3,2)

Generated Nodes

(3,3)

(4,2)

Queue

[(4,1),(3,3),(4,2)]

Step 10

Expand

(4,1)

No new node.

Queue

[(3,3),(4,2)]

Step 11

Expand

(3,3)

Generated Nodes

(3,4)

(4,3)

Queue
[(4,2),(3,4),(4,3)]

Step 12

Expand
(4,2)
No new node.
Queue
[(3,4),(4,3)]

Step 13

Expand
(3,4)
Generated Node
Goal (4,4)
Queue
[(4,3),(4,4)]

Step 14

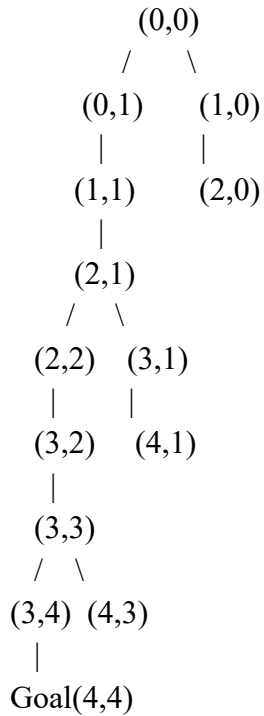
Expand
(4,4)
Goal Reached
Algorithm Terminates.

6. Queue (Frontier) Updates

Step Expanded Node Queue After Expansion

1	(0,0)	[(0,1),(1,0)]
2	(0,1)	[(1,0),(1,1)]
3	(1,0)	[(1,1),(2,0)]
4	(1,1)	[(2,0),(2,1)]
5	(2,0)	[(2,1)]
6	(2,1)	[(2,2),(3,1)]
7	(2,2)	[(3,1),(3,2)]
8	(3,1)	[(3,2),(4,1)]
9	(3,2)	[(4,1),(3,3),(4,2)]
10	(4,1)	[(3,3),(4,2)]
11	(3,3)	[(4,2),(3,4),(4,3)]
12	(4,2)	[(3,4),(4,3)]
13	(3,4)	[(4,3),(4,4)]
14	(4,4)	Goal Found

7. Search Tree



8. Manhattan Distance Heuristic

The Manhattan Distance is calculated using:

$$h(n) = |x_g - x_n| + |y_g - y_n|$$

where:

- $((x_g, y_g))$ = Goal coordinates
- $((x_n, y_n))$ = Current node coordinates

Heuristic Values

Node $h(n)$

(0,0) 8

(0,1) 7

(1,1) 6

(2,1) 5

(2,2) 4

(3,2) 3

(3,3) 2

(3,4) 1

(4,4) 0

Note: BFS does **not** use these values while selecting nodes. These values are provided only for comparison with informed search algorithms.

9. Optimal Path

(0,0)

↓

(0,1)

↓

(1,1)

↓

(2,1)

↓

(2,2)

↓

(3,2)

↓

(3,3)

↓

(3,4)

↓

(4,4)

10. Cost Calculation

Each movement costs 1.

Total Moves

8

Therefore,

$$\begin{aligned} [\\ \{ \text{Total} \quad \text{Cost} \} \quad = \quad 8 \quad \times \quad 1 \quad = \quad 8 \\] \end{aligned}$$

Optimal Path Cost = 8

11. Time and Space Complexity

Time Complexity

[
 $O(V+E)$
]

where:

- V = Number of vertices (grid cells)
- E = Number of edges (possible movements)

For a grid search, BFS visits each reachable cell at most once.

Space Complexity

[
 $O(V)$
]

because BFS stores visited nodes and the frontier queue.

12. Advantages of BFS

- Guarantees the shortest path when all edge costs are equal.
- Complete algorithm that finds a solution if one exists.
- Simple and easy to implement.
- Suitable for robot navigation, path planning, and maze solving.
- Ensures level-by-level exploration of the search space.

13. Limitations of BFS

- Requires large memory for storing the queue in large search spaces.
- Explores many unnecessary nodes before reaching the goal.
- Does not use heuristic information, making it slower than informed algorithms such as A* for complex environments.

14. Conclusion

The Breadth-First Search (BFS) algorithm successfully navigated the robot from the **Start (0,0)** to the **Goal (4,4)** in a 5×5 grid while avoiding obstacles. By exploring nodes level by level using a First-In, First-Out (FIFO) queue, BFS guaranteed the shortest path because all movements had equal cost. The algorithm expanded nodes systematically, maintained the frontier through queue updates, and reached the goal with an **optimal path cost of 8**. Although the Manhattan Distance heuristic was calculated, it did not influence node selection because BFS is an uninformed search algorithm. This demonstrates that BFS is an effective and optimal technique for uniform-cost grid navigation problems.

Question 3

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed

- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.**
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.**
- Demonstrate how the robot applies AI concepts such as:**
 - Intelligent agents
 - Rationality
 - Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.**

AI-Based Autonomous Rescue Robot for Disaster Response Using Uniform Cost Search and Online Search Agent

1. Introduction

Artificial Intelligence (AI) has become an essential technology in disaster management and emergency response systems. During natural disasters such as earthquakes, floods, landslides, and building collapses, rescue teams often face dangerous environments where human intervention is risky. Autonomous rescue robots equipped with Artificial Intelligence can safely explore hazardous areas, locate survivors, and assist rescue personnel while minimizing human risk. Unlike traditional robots that operate in completely known environments, rescue robots function in partially observable and dynamic environments where information is incomplete and changes continuously. The robot cannot view the entire building at once; instead, it discovers obstacles, risky zones, and safe paths as it moves. Therefore, intelligent decision-making and adaptive path planning are essential for successful navigation.

This problem models the disaster building as a grid environment where the rescue robot must travel from the Start (S) location to the Goal (G) while minimizing travel cost and avoiding dangerous areas. To solve this problem efficiently, Uniform Cost Search (UCS) combined with the behavior of an Online Search Agent is selected because it guarantees the lowest-cost path while continuously updating knowledge about the environment.

2. Problem Statement

An autonomous rescue robot is deployed inside a disaster-affected building to locate trapped survivors.

The building consists of multiple rooms arranged as a grid.

The robot

- Starts from S
- Must reach the survivor located at G
- Has incomplete knowledge of the environment
- Must avoid blocked paths

- Must minimize travel cost
- Must continuously update its knowledge while exploring

The objective is to safely rescue survivors while consuming minimum resources and avoiding unnecessary risks.

3. Problem Representation

Variables

Robot Position

Current Cost

Visited Cells

Risk Level

Obstacle Information

Knowledge Base

State Space

Each room in the building represents one state.

The robot changes its state whenever it moves to a neighbouring room.

Initial State

Robot located at

Start (S)

Goal State

Robot reaches

Goal (G)

where the survivor is located.

4. Environment Representation

Disaster Building

S □ □ X □

□ R □ X □

□ □ □ □ □

X □ R □ G

□ □ □ □ □

Legend

S → Start

G → Goal

□ → Safe Room

X → Obstacle

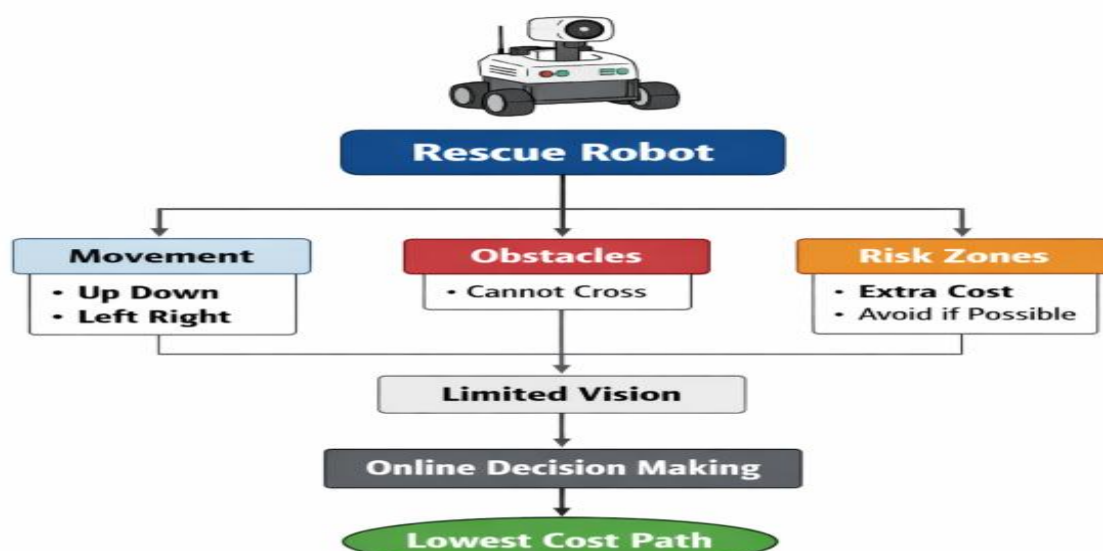
R → Risk Zone

5. Constraint Analysis

The constraints directly influence the robot's navigation strategy and decision-making capability. Each constraint must be analyzed carefully before selecting an appropriate search algorithm.

Constraint	Analysis	Effect on Decision Making
Robot can move Up, Down, Left and Right	Limits movement to four neighbouring cells. Diagonal movement is prohibited.	The robot must evaluate only valid neighbouring rooms before selecting the next action.
Obstacles cannot be crossed	Blocked rooms are permanently inaccessible.	The robot removes blocked cells from the search space and computes alternative paths.
Limited sensing capability	The robot can observe only adjacent rooms.	Global path planning is impossible; the robot must update its map incrementally.
Movement cost = 1	Every movement consumes energy and time.	The robot prefers shorter paths whenever possible.
Risk Zone Cost = 2	Entering hazardous areas increases total travel cost.	The robot avoids risky rooms unless they are necessary to reach the survivor.

6. Constraint Relationship Diagram



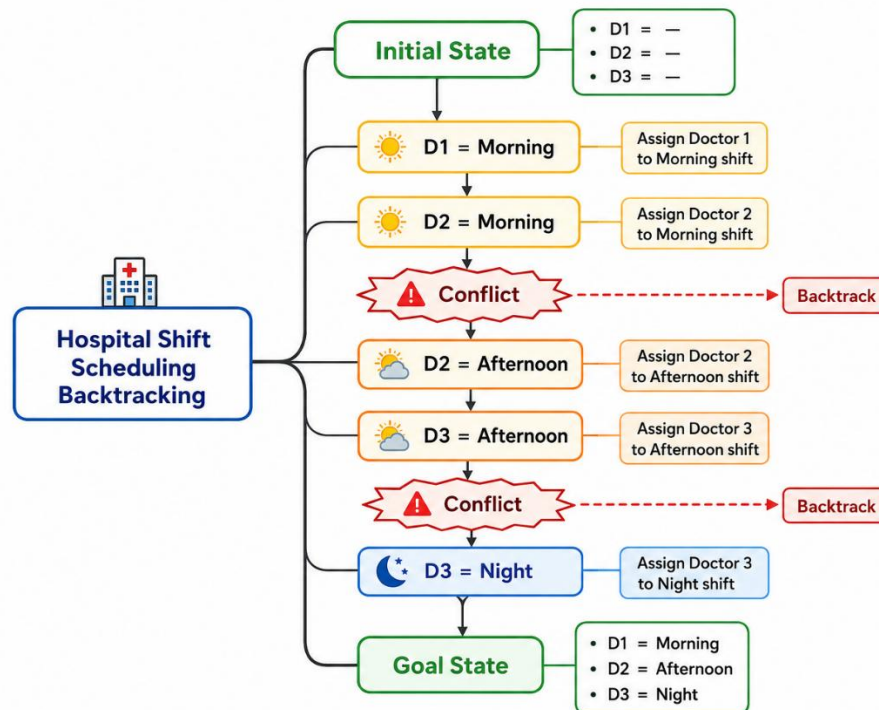
7. Selection of Search Strategy

The most suitable search strategy is

Uniform Cost Search (UCS) with Online Search Agent

Why UCS?

- Uniform Cost Search always expands the node having the **minimum cumulative path cost** instead of the shortest distance.
- Since entering risky rooms increases the travel cost, UCS naturally prefers safer paths even if they require more movements.
- The Online Search Agent complements UCS by allowing the robot to update its knowledge whenever newly discovered obstacles or hazards appear.



8. Uniform Cost Search Algorithm

Step 1

Initialize the priority queue.

Insert the Start node.

Step 2

Remove the node having the lowest cumulative cost.

Step 3

Observe neighbouring rooms.

Step 4

Ignore blocked cells.

Step 5

Calculate new path cost.

Safe Room
Cost +1
Risk Zone
Cost +3

Step 6

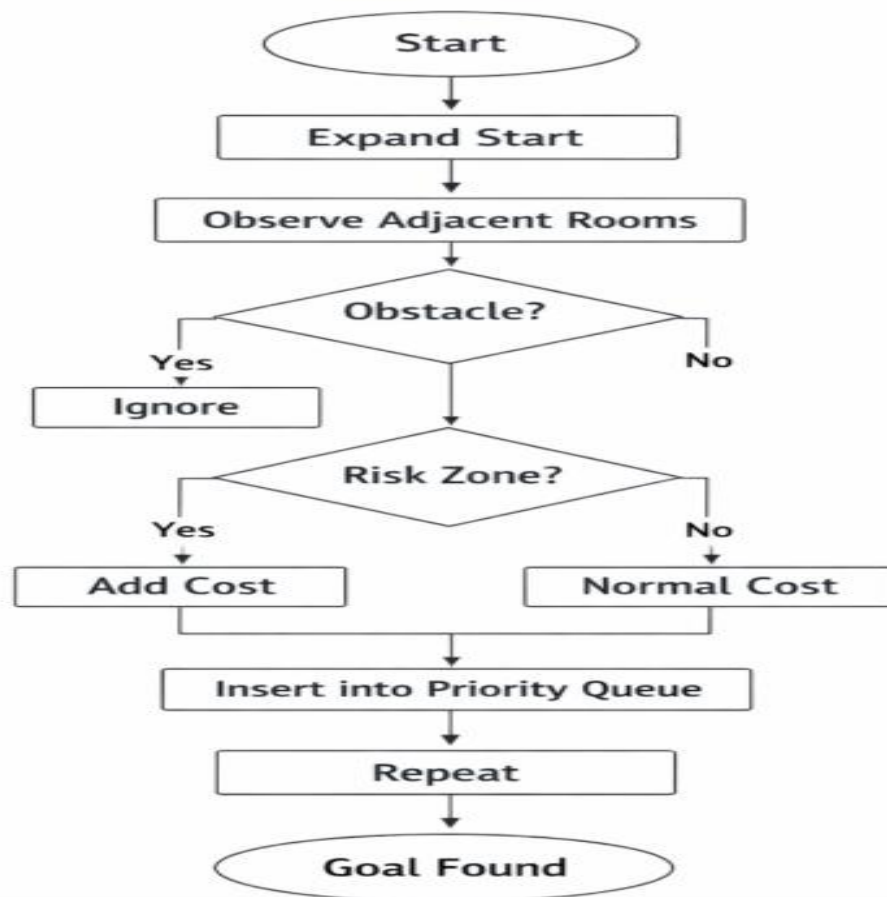
Insert newly discovered rooms into the priority queue.

Step 7

Repeat until Goal is reached.

9. Search Process

Search Algorithm For Exploring An Unknown Building



10. Online Search Behaviour

Unlike offline search, the robot does not know the complete environment initially.

During movement,

- New obstacles may appear.
- Risk levels may change.
- Previously unknown rooms become visible.

The robot immediately updates its internal map and recalculates the best path.

This adaptive behaviour improves safety and increases rescue success.

11. Application of Artificial Intelligence Concepts

(a) Intelligent Agent

The rescue robot acts as an intelligent agent because it

- observes its environment,
- analyses available information,
- selects appropriate actions,
- learns new information,
- reaches the goal autonomously.

(b) Rationality

A rational agent always chooses the action that maximizes performance.

The rescue robot

- ✓ minimizes travel cost
- ✓ avoids risky zones
- ✓ safely reaches survivors
- ✓ updates decisions whenever new information becomes available

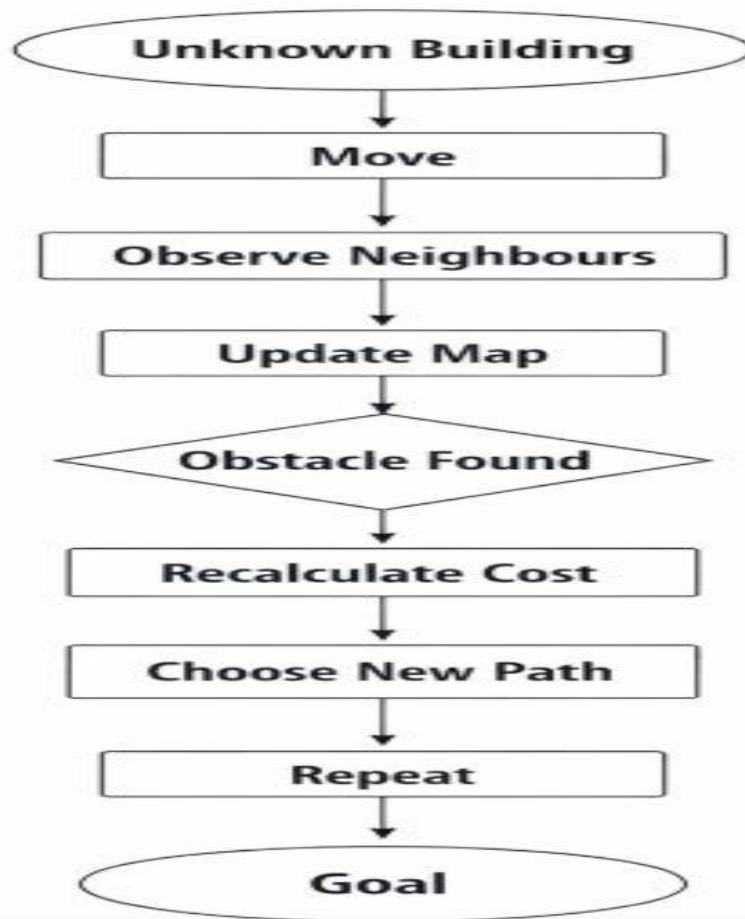
Therefore, every movement is based on rational decision making.

(c) Environment Type

Property	Description
Observable	Partially Observable
Agents	Single Agent
Deterministic	Partially Deterministic
Dynamic	Dynamic Environment
State Space	Sequential
Knowledge	Unknown Initially

The robot must continuously learn because the environment is only partially observable.

12. Knowledge Updating Process



13. Advantages of UCS + Online Search

- Always finds the minimum-cost solution.
- Avoids unnecessary risky zones.
- Adapts to changing environments.
- Efficient for disaster response.
- Supports real-time navigation.
- Reduces rescue time.
- Conserves battery power.
- Improves survivor detection efficiency.

14. Real-Time Applications

This approach is widely used in

- Earthquake rescue robots
- Firefighting robots
- Military surveillance robots
- Space exploration rovers
- Autonomous warehouse robots
- Search-and-rescue drones
- Smart disaster management systems
- Industrial inspection robots

15. Justification

Uniform Cost Search combined with an Online Search Agent is the most suitable solution because it satisfies all given constraints. UCS guarantees that the robot always expands the path with the lowest accumulated cost, ensuring that risky zones are entered only when unavoidable. At the same time, the Online Search Agent enables the robot to operate effectively in a partially observable and dynamic environment, continuously updating its internal knowledge base as new information is discovered. This combination results in intelligent, rational, and adaptive navigation that minimizes travel cost while maximizing safety and rescue efficiency. Hence, it is an ideal AI approach for autonomous disaster-response robots.

16. Conclusion

The proposed AI-Based Autonomous Rescue Robot demonstrates how Artificial Intelligence techniques can solve complex real-world navigation problems in disaster environments. By integrating Uniform Cost Search with an Online Search Agent, the robot intelligently explores unknown areas, dynamically updates its knowledge, avoids obstacles, minimizes traversal through risky zones, and identifies the safest path to the survivor. The solution effectively applies the principles of intelligent agents, rational decision-making, and dynamic environment handling, making it a robust, efficient, and practical approach for emergency search-and-rescue operations. Its ability to optimize safety, reduce operational cost, and adapt to changing conditions makes it highly suitable for modern disaster management systems.